

Daily DSA / Backend Practice Notes

Python & SQL — Lists of Dictionaries, Copying, Sorting, and GROUP BY / HAVING

Python

Problem Solved

Processed a list of student dictionaries by:

- Calculating average marks.
- Adding **average** and **result**.
- Creating a new list instead of modifying the original.
- Sorting by average.
- Printing students with at least one mark > 90.

Concepts Learned

1. Functions

```
def avg(marks):  
    return sum(marks) / len(marks)
```

- Don't hardcode values like /3.
- Write reusable functions.

2. *args

Not required when the function already receives a list.

Instead of

```
def avg(*args):
```

Use

```
def avg(marks):
```

3. Assignment vs Shallow Copy vs Deep Copy

Assignment

```
new_student = student
```

- Same object.
- Any modification affects both variables.

Shallow Copy

```
new_student = student.copy()
```

- Creates a new dictionary.
- Nested lists/dictionaries are still shared.

Example

```
new_student["average"] = 90
```

■ Original unchanged

```
new_student["marks"].append(100)
```

■ Original changes too.

Deep Copy

```
import copy

new_student = copy.deepcopy(student)
```

- Everything is copied.
- Nested objects are independent.

4. Sorting

New List

```
sorted(data, key=lambda x: x["average"], reverse=True)
```

Returns a new sorted list.

In-place Sorting

```
data.sort(key=lambda x: x["average"], reverse=True)
```

Sorts the existing list.

5. any()

Check whether at least one element satisfies a condition.

```
if any(mark > 90 for mark in student["marks"]):
```

6. Safe Average Function

Avoid division by zero.

```
def avg(marks):
    if not marks:
        return 0
    return sum(marks) / len(marks)
```

7. Dictionary Values

When marks are stored as:

```
{
    "Math": 90,
    "Physics": 85
}
```

Average

```
sum(student["marks"].values()) / len(student["marks"])
```

Use `.values()`, not `.keys()`.

Real-world Usage

List of dictionaries appears in:

- REST API responses
- JSON files
- MongoDB documents
- Pandas (DataFrame creation)
- ETL pipelines
- LLM messages (role, content)

SQL

Problem Solved

Grouped customer orders to calculate:

- Total Orders
- Total Amount
- Average Amount

Filtered customers having at least 2 orders. Sorted by total amount.

Final query

```
SELECT customer_id,  
       COUNT(order_id) AS total_orders,  
       SUM(amount) AS total_amount,  
       AVG(amount) AS avg_amount  
FROM Orders  
GROUP BY customer_id  
HAVING COUNT(*) >= 2  
ORDER BY total_amount DESC;
```

Concepts Learned

Aggregate Functions

```
COUNT()  
SUM()  
AVG()
```

GROUP BY

Creates groups before applying aggregate functions.

HAVING

Filters groups.

Example

```
HAVING COUNT(*) >= 2
```

WHERE vs HAVING

WHERE

- Filters rows.
- Executes before grouping.

HAVING

- Filters groups.
- Executes after grouping.

COUNT(*) vs COUNT(column)

- COUNT (*) counts every row.
- COUNT (column) counts only non-NULL values.

SQL Execution Order (memorize this)

Memorize this:

```
FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY
LIMIT
```

Mnemonic: From Where Groups Have Selected Ordered Limits

Common Mistakes Corrected Today

Python

✗ Hardcoded average	✓ Used len(marks)
✗ Modified original list	✓ Used student.copy()
✗ Didn't sort	✓ Used sorted(..., reverse=True)

SQL

✗ HAVING COUNT(*) > 2	✓ HAVING COUNT(*) >= 2
✗ ORDER BY total_amount;	✓ ORDER BY total_amount DESC;

Homework (5 Minutes)

Python

Revise:

- copy()
- deepcopy()
- sorted()
- .sort()

- any()

SQL

Memorize:

```
FROM  
WHERE  
GROUP BY  
HAVING  
SELECT  
ORDER BY  
LIMIT
```